

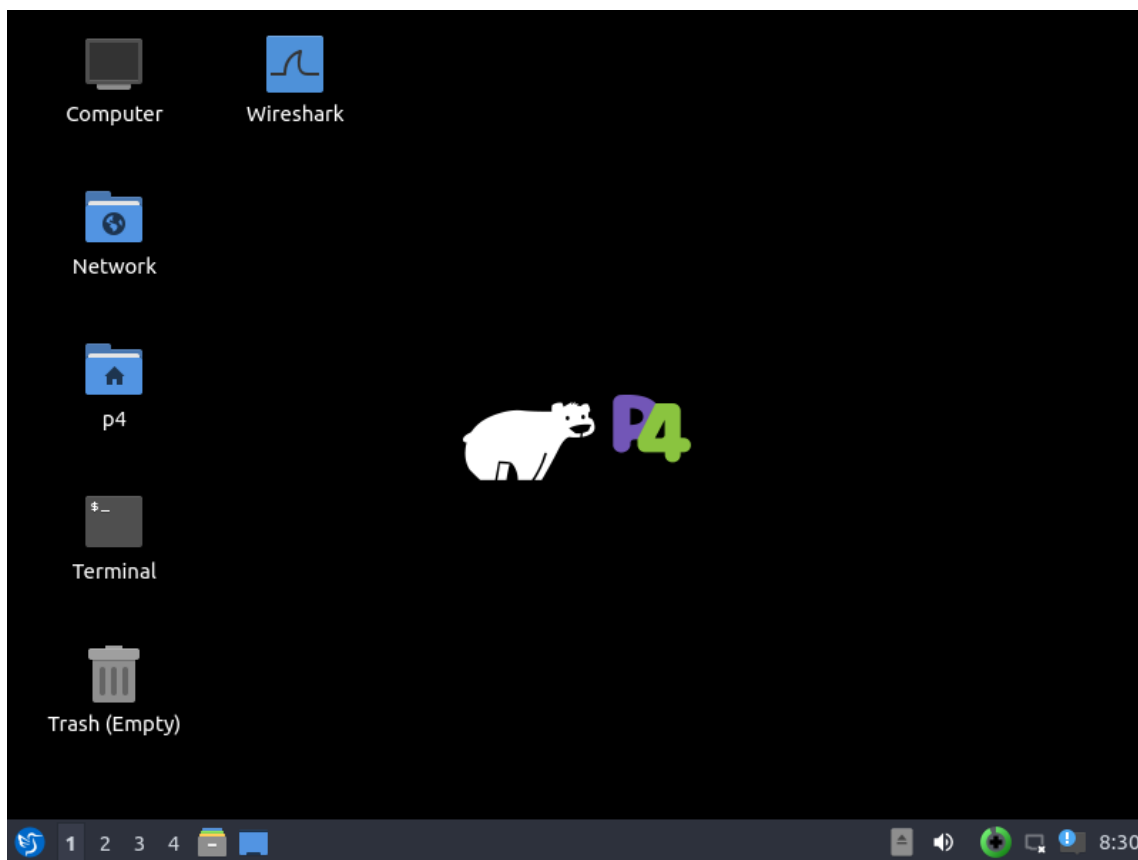
Ćwiczenie 3

p4runtime

źródło: github.com/p4lang/tutorials

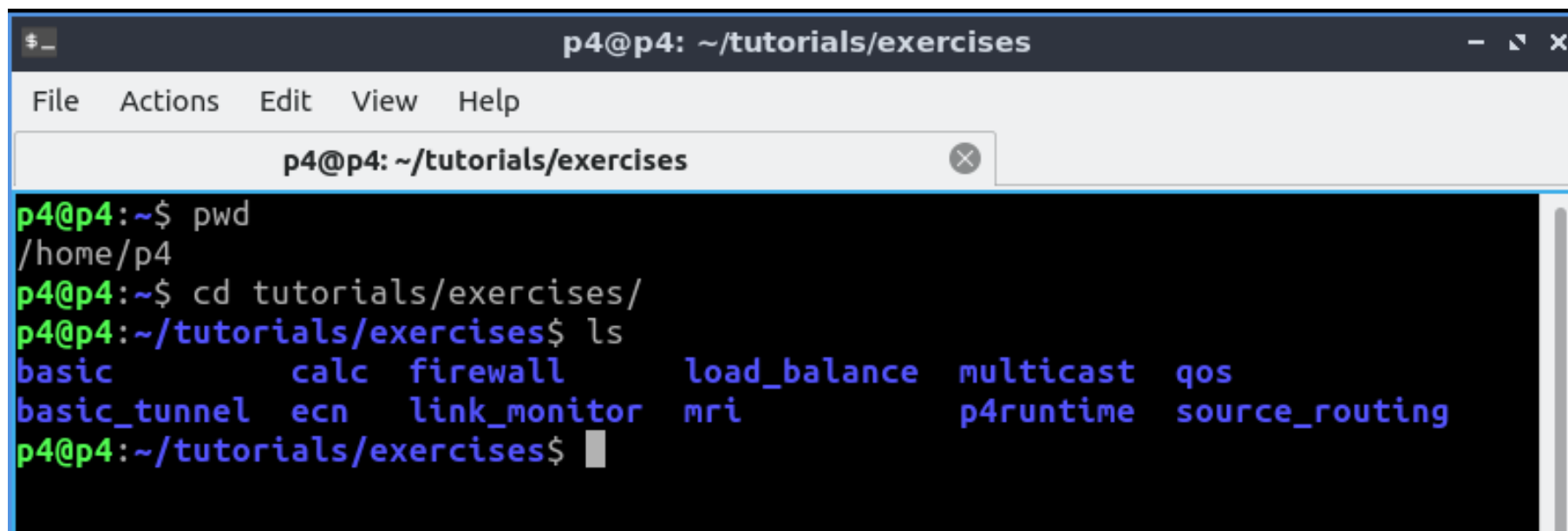
Przygotowanie - VM

- uruchamiamy maszynę P4 Tutorial (w VirtualBox)
 - dla ułatwienia:
 - login: P4
 - hasło: p4
- ruszyła maszyna (?)
wcale nie ospale 😊
- czego potrzebujemy:
 - terminala
 - przejścia do katalogu tutorials/exercises



Co znajduje się w katalogu z ćwiczeniami

- p4runtime: implementacja prostego (w sensie funkcjonalności na pewno) kontrolera switcha P4
- inne...



```
p4@p4: ~/tutorials/exercises
File  Actions  Edit  View  Help
p4@p4: ~/tutorials/exercises
p4@p4:~$ pwd
/home/p4
p4@p4:~$ cd tutorials/exercises/
p4@p4:~/tutorials/exercises$ ls
basic          calc  firewall      load_balance  multicast  qos
basic_tunnel  ecn   link_monitor  mri           p4runtime  source_routing
p4@p4:~/tutorials/exercises$
```

Struktura katalogu z ćwiczeniem p4runtime.p4

```
p4@p4:~/tutorials/exercises/p4runtime$ ls
advanced_tunnel.p4  mycontroller.py  solution
Makefile           README.md       topology.json
p4@p4:~/tutorials/exercises/p4runtime$
```

- README.md : opis ćwiczenia
- advanced_tunnel.p4 :
program w P4 opisujący zachowanie urządzenia
- mycontroller.py : kontroler korzystający z P4Runtime
 - tu będziemy działać
- Makefile

Krok 0: cel ćwiczenia, co mamy

- spodziewanym efektem realizacji ćwiczenia jest implementacja prostego kontrolera P4, który potrafi
 - wypełniać tablice switcha,
 - odczytywać zawartość tablic sterujących urządzeniem,
 - odczytywać wartości liczników
- ćwiczenie korzysta z topologii trójkąta
 - UWAGA: wartości `dst_id` są inne, niż w poprzednim ćwiczeniu

Krok 0: cel ćwiczenia, co mamy

- `advanced_tunnel.p4` zawiera:
 - definicję dwóch tablic liczników
 - każdy możliwy tunel jest wyposażony w licznik i we/wy pakietów i bajtów

```
control MyIngress (...) {  
  ...  
  counter (MAX_TUNNEL_ID, CounterType.packets_and_bytes) ingressTunnelCounter;  
  counter (MAX_TUNNEL_ID, CounterType.packets_and_bytes) egressTunnelCounter;  
  ...  
}
```

- definicję dwóch akcji:
 - `myTunnel_ingress` dodaje nagłówek tunelowy i zwiększa licznik
 - `myTunnel_egress` zdejmuje nagłówek i zwiększa licznik

Krok 1: make run

- make run
 - kompiluje program i wchodzi do środowiska Mininet
- h1 ping h2
 - zostawiamy
- w drugim terminalu (ale w tej samej lokalizacji): ./mycontroller.py

```
----- Reading tunnel counters -----  
s1 MyIngress.ingressTunnelCounter 100: 4 packets (392 bytes)  
s2 MyIngress.egressTunnelCounter 100: 0 packets (0 bytes)  
s2 MyIngress.ingressTunnelCounter 200: 0 packets (0 bytes)  
s1 MyIngress.egressTunnelCounter 200: 0 packets (0 bytes)
```

- obserwacja: coś działa – zwiększa się licznik pakietów próbujących wejść do tunelu
- Ctrl-C, make stop, make clean ... i do roboty!

Krok 2: uzupełnianie

- co dzieje się w pliku mycontroller.py:
 - tworzone są połączenia gRPC do s1 i s2

```
def main(p4info_file_path, bmv2_file_path):  
...  
    s1 = p4runtime_lib.bmv2.Bmv2SwitchConnection( ... )  
    s2 = p4runtime_lib.bmv2.Bmv2SwitchConnection( ... )  
...
```

- instalowany jest program P4 na switchach

```
s1.MasterArbitrationUpdate()  
s1.SetForwardingPipelineConfig(p4info,bmv2_file_path)  
...
```

Krok 2: uzupełnianie

- co dzieje się w pliku mycontroller.py:
 - wpisywane są reguły do tablic

```
writeTunnelRules (  
    p4info_helper, ingress_sw=s1, egress_sw=s2, tunnel_id=100,  
    dst_eth_addr="08:00:00:00:02:22", dst_ip_addr="10.0.2.2")
```

Jakie reguły tworzone są
przez te wywołania?

- odczytywane są liczniki dla tuneli 100 i 200

```
printCounter(  
    p4info_helper, s1, "MyIngress.ingressTunnelCounter", 100)  
...  
printCounter(  
    p4info_helper, s2, "MyIngress.ingressTunnelCounter", 200)  
...
```

Co powinna robić brakująca akcja?

- na s1:
ramki z etykietą 100 powinny być przekazywane
na port łączący go z s2 (2)
- na s2: ?
- szczęśliwie – taką akcję mamy zdefiniowaną(!)
 - ... ale – gdzie?

```
action myTunnel_forward(egressSpec_t port) {  
    standard_metadata.egress_spec = port;  
}
```

Dodajemy własny wpis do tablicy (dynamicznie!)

- tablica: `myTunnel_exact`
- klucz: `dst_id`
- wartość: akcja `myTunnel_forward`
 - argument: numer portu wyjściowego
- całość instalujemy na switchu będącym wejściem do tunelu

```
table_entry = p4info_helper.buildTableEntry(  
    table_name="MyIngress.myTunnel_exact",  
    match_fields={  
        "hdr.myTunnel.dst_id": tunnel_id  
    },  
    action_name="MyIngress.myTunnel_forward",  
    action_params={  
        "port": SWITCH_TO_SWITCH_PORT  
    })  
  
ingress_sw.WriteTableEntry(table_entry)
```

Krok 3: **make** (it) **run** again

- Zaczyna działać!

```
----- Reading tunnel counters -----  
s1 MyIngress.ingressTunnelCounter 100: 2 packets (196 bytes)  
s2 MyIngress.egressTunnelCounter 100: 2 packets (204 bytes)  
s2 MyIngress.ingressTunnelCounter 200: 2 packets (196 bytes)  
s1 MyIngress.egressTunnelCounter 200: 2 packets (204 bytes)
```

- Kilka pytań:
 - dlaczego liczniki bajtów wchodzących i wychodzących mają różne wartości?
 - ile mamy zdefiniowanych liczników?
 - w jaki sposób zrealizowane jest dodawanie i zdejmowanie nagłówka?

Krok 4: systematyzujemy

- fragmenty zawartości `advanced_tunnel.p4`

```
action myTunnel_ingress(bit<16> dst_id) {  
    hdr.myTunnel.setValid();  
    hdr.myTunnel.dst_id = dst_id;  
    hdr.myTunnel.proto_id = hdr.ethernet.etherType;  
    hdr.ethernet.etherType = TYPE_MYTUNNEL;  
    ingressTunnelCounter.count((bit<32>) hdr.myTunnel.dst_id);  
}
```

```
action myTunnel_egress(macAddr_t dstAddr, egressSpec_t port) {  
    standard_metadata.egress_spec = port;  
    hdr.ethernet.dstAddr = dstAddr;  
    hdr.ethernet.etherType = hdr.myTunnel.proto_id;  
    hdr.myTunnel.setInvalid();  
    egressTunnelCounter.count((bit<32>) hdr.myTunnel.dst_id);  
}
```

- czy te akcje są uwzględniane w definicji tablic? Których?

Krok 5: dodatki

- (dodatek 1) Kontroler może nie tylko dokonywać wpisów, ale również odczytywać zawartości tablic
 - w kodzie kontrolera odkomentujemy wywołanie `readTableRules()` (x2)
 - efekt?
- (dodatek 2) W kodzie zawartym w katalogu `solution/` znajdujemy tłumaczenie kodów na nazwy i zapisu ósemkowego na szesnastkowy
 - wskazówka: „extra credit”

Ćwiczenie 4: ustawianie bitów w nagłówku IP

Explicit Congestion Notification (ecn)

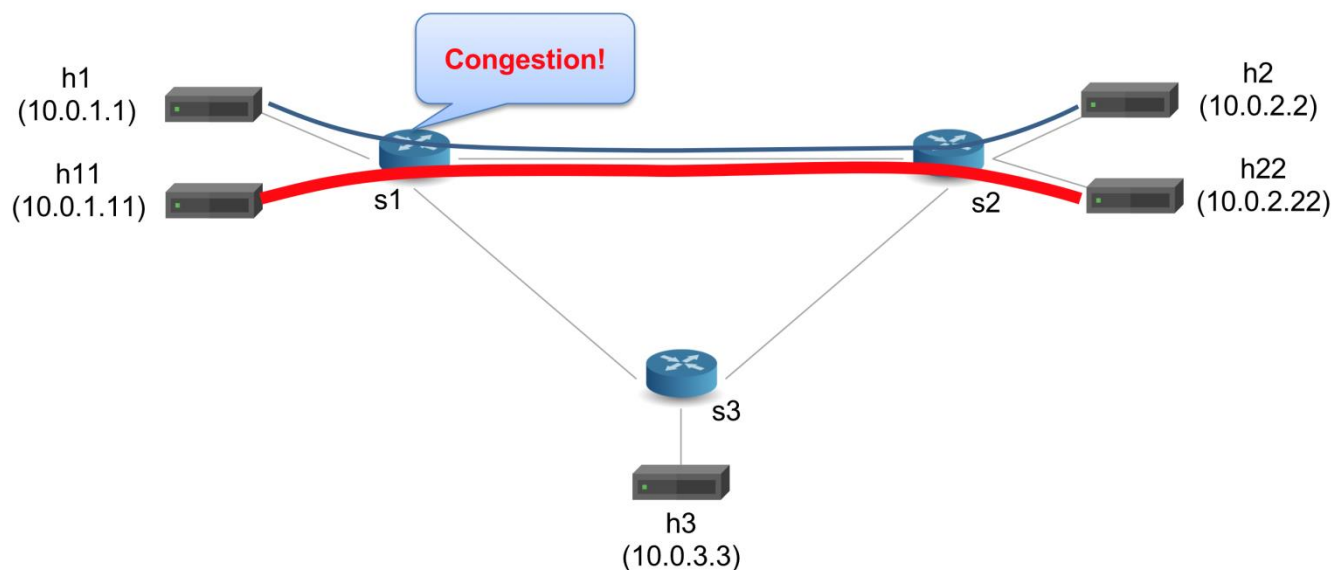
źródło: github.com/p4lang/tutorials

ECN – co to takiego i do czego służy

Struktura katalogu z ćwiczeniem ecn.p4

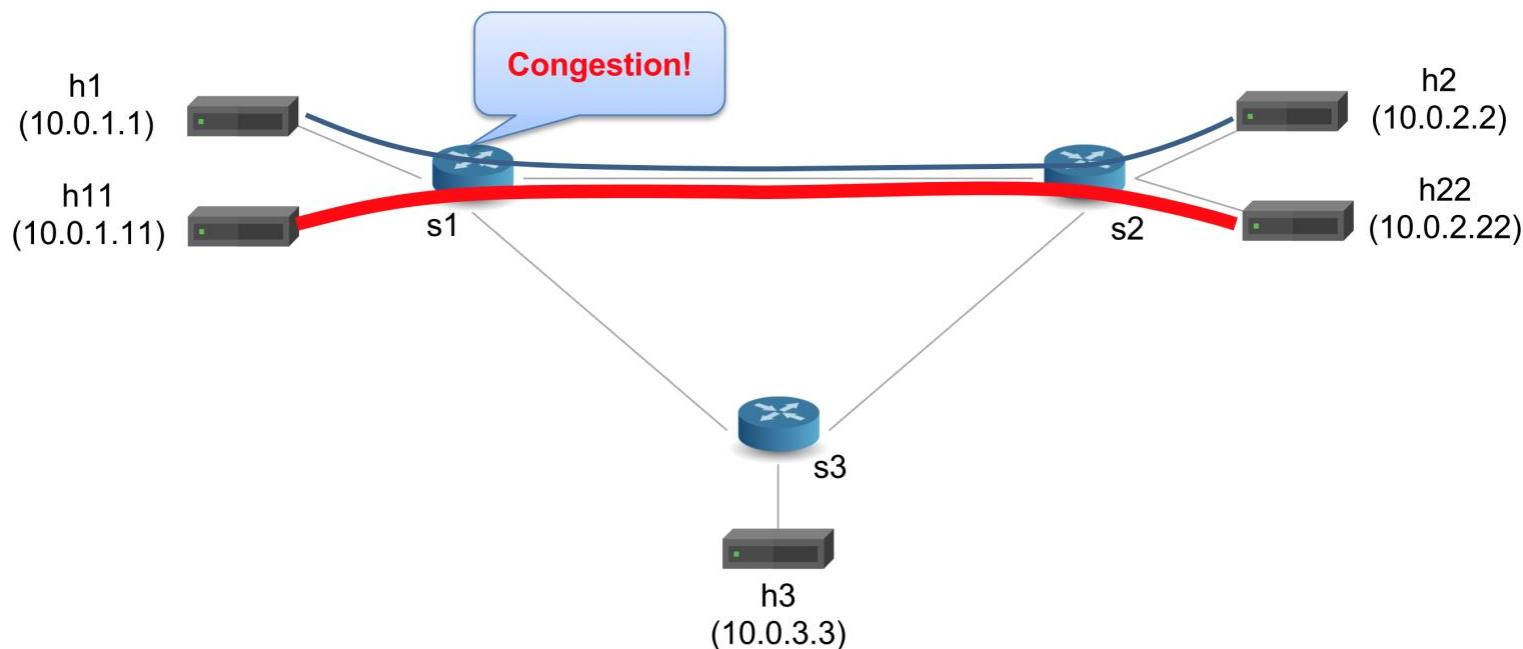
```
p4@p4:~/tutorials/exercises/ecn$ ls
ecn.p4      README.md    s1-runtime.json  s3-runtime.json  setup.png  topology.json
Makefile    receive.py   s2-runtime.json  send.py          solution
```

- README.md : opis ćwiczenia
- ecn.p4 :
program w P4 opisujący zachowanie switcha
- topology.json: opis topologii
- Makefile



Krok 0: co mamy

- Sprawdzamy zawartość plików:
 - topology.json
 - topologia trójkąta uzupełniona jest o dwa hosty
 - łącze s1-s2 ma niską przepustowość
 - s*-runtime.json
 - reguły przekazywania pakietów są już wypełnione



Krok 1: pakiety - biegajcie!

- make run
- xterm h1 h2 h11 h22
- uruchamiamy przepływ danych między h1 i h2
 - h2: ./receive.py
 - h1: ./send.py 10.0.2.2 "AGH rulez" 200
- uruchamiamy przepływ dociążający łącze między h11 i h22
 - h22: iperf -s -u
 - h11: iperf -c 10.0.2.22 -t 15 -u
- Obserwacja: pole *type of service* (a właściwie DSCP+ECN) cały czas ma wartość 1 ← to chcemy zmienić

Krok 2: modyfikujemy zachowanie switcha

- ecn.p4:

- /* TODO: split tos to two fields 6 bit diffserv and 2 bit ecn */

```
bit<8>    tos;
```



```
bit<6>    diffserv;  
bit<2>    ecn;
```

- zmiana nagłówka skutkuje koniecznością zmiany wszystkich odwołań do niego, w szczególności przy obliczaniu sumy kontrolnej

```
hdr.ipv4.tos,
```



```
hdr.ipv4.diffserv,  
hdr.ipv4.ecn,
```

Krok 2: modyfikujemy zachowanie switcha

- ecn.p4: w bloku MyEgress()
 - `/* TODO: compare standard_metadata.enq_qdepth with threshold*/`

```
action mark_ecn() {  
    hdr.ipv4.ecn = 3;  
}  
apply {  
    if (hdr.ipv4.ecn == 1 || hdr.ipv4.ecn == 2) {  
        if (standard_metadata.enq_qdepth >= ECN_THRESHOLD) {  
            mark_ecn();  
        }  
    }  
}
```

- `ECN_THRESHOLD`: stała zdefiniowana na początku programu (o wartości 10)

Krok 3: obserwujemy

- make run
- xterm h1 h2 h11 h22
- uruchamiamy przepływ danych między h1 i h2
 - h2: `./receive.py | grep tos`
 - h1: `./send.py 10.0.2.2 "AGH rulez" 200`
- uruchamiamy przepływ dociążający łącze między h11 i h22
 - h22: `iperf -s -u`
 - h11: `iperf -c 10.0.2.22 -t 15 -u`
- Obserwacja: pole *type of service* (a właściwie DSCP+ECN) ma wartość 1, potem 3, potem znowu 1

Ćwiczenie 5: własna opcja IP

Multi-hop route inspection (MRI)

źródło: github.com/p4lang/tutorials

Krok 0: cel ćwiczenia, struktura katalogu

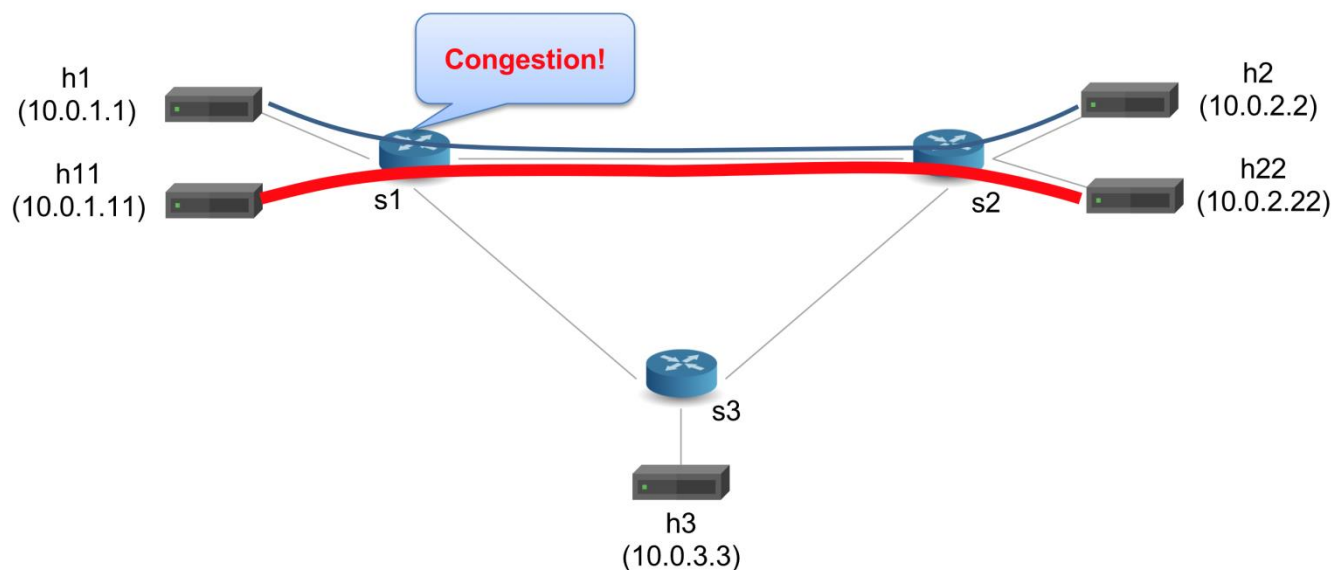
- Celem ćwiczenia jest implementacja prostego mechanizmu monitorującego, pozwalającego na zebranie poziomów zajętości kolejek na urządzeniach sieciowych znajdujących się na trasie pakietu (uproszczona wersja INT)
- Sposób działania: doklejanie informacji o stanie zapełnienia do nagłówka pakietu IP w formie nowej opcji
- Struktura katalogu:

```
p4@p4:~/tutorials/exercises/mri$ ls
Makefile  README.md  s1-runtime.json  s3-runtime.json  setup.png  topology.json
mri.p4    receive.py  s2-runtime.json  send.py          solution
```

- mri.p4: tu będziemy działać
- inne – jak w poprzednich przykładach

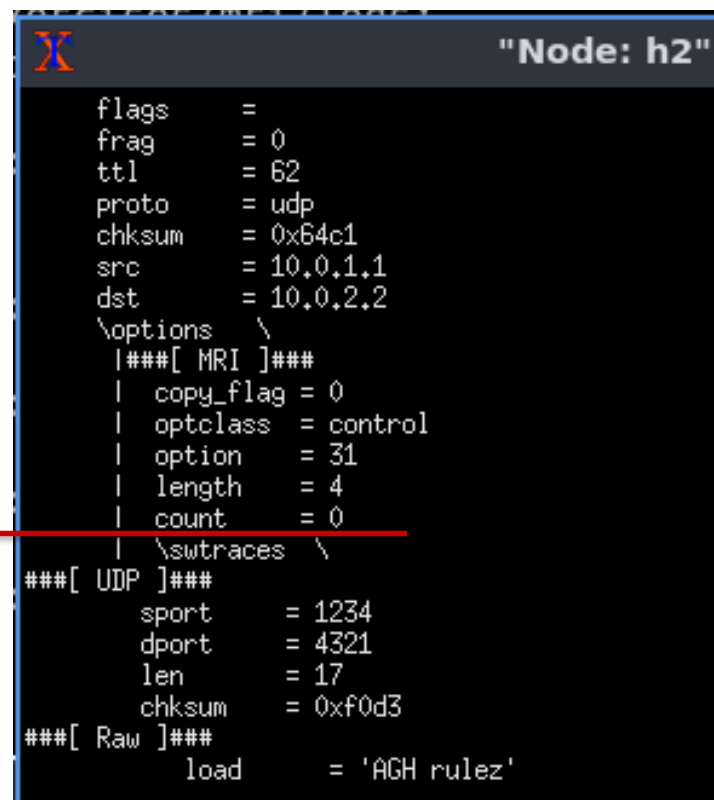
Topologia, opis eksperymentu

- Łącze pomiędzy s1 a s2 cechuje się niską przepustowością
- Dwa przepływy (h1-h2 oraz h11-h22) współdzielą to łącze
- Przepływ h11-h22 mocno obciąża łącze
- Cel: zaobserwować zwiększoną zajętość buforów



Krok 1: co działa, czego brakuje

- make run
- xterm h1 h11 h2 h22
- h2: ./receive.py
- h1: ./send.py 10.0.2.2 "AGH rulez" 200
- h22: iperf -s -u
- h11: iperf -c 10.0.2.22 -t 15 -u
- Obserwacja: na h2 dochodzą pakiety bez zarejestrowanej trasy (count = 0)



```
Node: h2
flags      =
frag       = 0
ttl        = 62
proto      = udp
chksum     = 0x64c1
src        = 10.0.1.1
dst        = 10.0.2.2
\options   \
|###[ MRI ]###
|  copy_flag = 0
|  optclass  = control
|  option    = 31
|  length    = 4
|  count     = 0
|  \swtraces \
###[ UDP ]###
      sport   = 1234
      dport   = 4321
      len      = 17
      chksum   = 0xf0d3
###[ Raw ]###
      load     = 'AGH rulez'
```

Nowe elementy

- Opcja, pola, nagłówki, błąd, stałe

```
typedef bit<32> switchID_t;
typedef bit<32> qdepth_t;
[...]
header ipv4_option_t {
    bit<1> copyFlag;
    bit<2> optClass;
    bit<5> option;
    bit<8> optionLength;
}

header mri_t {
    bit<16> count;
}

header switch_t {
    switchID_t swid;
    qdepth_t qdepth;
}
```

```
struct headers {
    ethernet_t      ethernet;
    ipv4_t          ipv4;
    ipv4_option_t   ipv4_option;
    mri_t           mri;
    switch_t[MAX_HOPS] swtraces;
}

error { IPHeaderTooShort }
```

```
const bit<5> IPV4_OPTION_MRI = 31;

#define MAX_HOPS 9
```

Nowe elementy

- Metadane

```
struct ingress_metadata_t {  
    bit<16>  count;  
}  
  
struct parser_metadata_t {  
    bit<16>  remaining;  
}  
  
struct metadata {  
    ingress_metadata_t    ingress_metadata;  
    parser_metadata_t     parser_metadata;  
}
```

W poprzednich ćwiczeniach
ta struktura była pusta

Nowe elementy

- Odczyt opcji IPv4 w parserze ...

```
state parse_ipv4 {  
    packet.extract(hdr.ipv4);  
    verify(hdr.ipv4.ihl >= 5, error.IPHeaderTooShort);  
    transition select(hdr.ipv4.ihl) {  
        5                : accept;  
        default          : parse_ipv4_option;  
    }  
}
```

- ... do uzupełnienia

Krok 2: uzupełniamy

- Parsujemy opcję MRI

```
packet.extract(hdr.ipv4_option);  
    transition select(hdr.ipv4_option.option) {  
        IPV4_OPTION_MRI : parse_mri;  
        _ : accept;  
    }
```

```
packet.extract(hdr.mri);  
    meta.parser_metadata.remaining = hdr.mri.count;  
    transition select(meta.parser_metadata.remaining) {  
        0: accept;  
        default: parse_swtrace;  
    }
```

```
packet.extract(hdr.swtraces.next);  
    meta.parser_metadata.remaining = meta.parser_metadata.remaining-1;  
    transition select( meta.parser_metadata.remaining ) {  
        0 : accept;  
        _ : parse_swtrace;  
    }
```

Krok 2: uzupełniamy

- Dopisujemy się (jako switch) do zawartości opcji MRI

```
hdr.mri.count = hdr.mri.count+1;
hdr.swtraces.push_front(1);
hdr.swtraces[0].setValid();
hdr.swtraces[0].swid = swid;
hdr.swtraces[0].qdepth = (qdepth_t)standard_metadata.deq_qdepth;
hdr.ipv4.ihl = hdr.ipv4.ihl+2;
hdr.ipv4.totalLen = hdr.ipv4.totalLen+8;
hdr.ipv4_option.optionLength = hdr.ipv4_option.optionLength+8;
```

```
if(hdr.mri.isValid()) swtrace.apply();
```

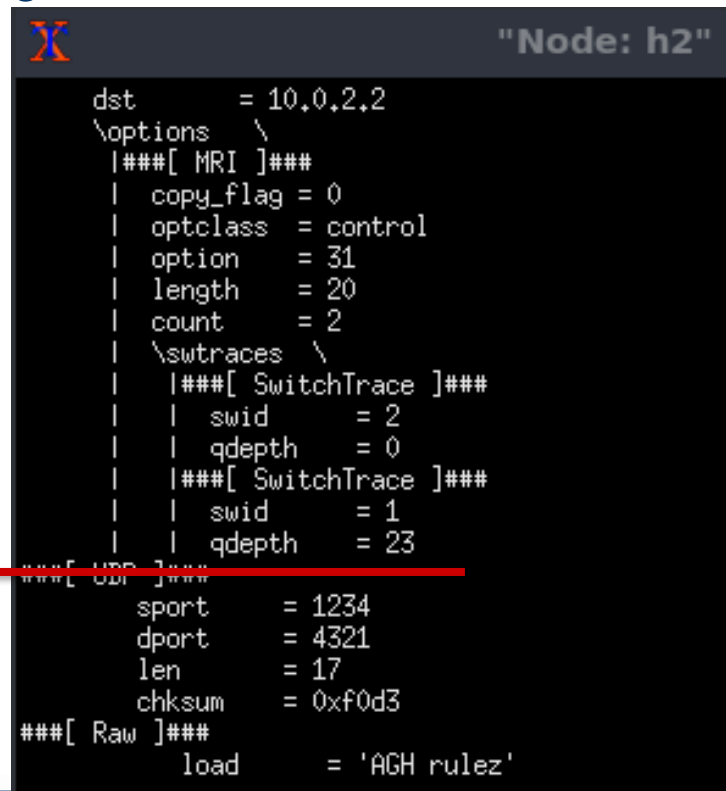
... ale tylko do pakietów, które taką opcję przenoszą

- i emitujemy pakiet

```
packet.emit(hdr.ipv4_option);
packet.emit(hdr.mri);
packet.emit(hdr.swtraces);
```

Krok 3: obserwacja wyników

- make run
- xterm h1 h11 h2 h22
- h2: ./receive.py
- h1: ./send.py 10.0.2.2 "AGH rulez" 200
- h22: iperf -s -u
- h11: iperf -c 10.0.2.22 -t 15 -u
- Obserwacja: na h2 dochodzą pakiety z zarejestrowaną trasą oraz z licznikami pakietów w kolejce (na łączy s1-s2 w trakcie działania iperf będą niezerowe)



```
Node: h2
dst      = 10.0.2.2
\options \
|###[ MRI ]###
|  copy_flag = 0
|  optclass  = control
|  option    = 31
|  length    = 20
|  count     = 2
|  \swtraces \
|  |###[ SwitchTrace ]###
|  |  swid     = 2
|  |  qdepth   = 0
|  |###[ SwitchTrace ]###
|  |  swid     = 1
|  |  qdepth   = 23
|###[ UDP ]###
|  sport      = 1234
|  dport      = 4321
|  len        = 17
|  chksum     = 0xf0d3
|###[ Raw ]###
|  load       = 'AGH rulez'
```

na dzisiaj tyle, tzn. dziękuję za uwagę

Kanał P4 Consortium na YT:

<https://www.youtube.com/channel/UCOQAFkDKucJWr-KafdJsdIQ>